

Compilation

Le langage Hopix

Adrien Guatto

Master 1 Informatique
Semestre 1 – 2025

Le langage Hopix

Les grandes lignes

- Fonctionnel : fonctions anonymes, d'ordre supérieur ; “orienté valeur”
- Impératif : évaluer une expression produit une valeur et *a un effet*.
- Typé : les programmes mal typés sont rejetés *avant* l'exécution.

Le langage Hopix

Les grandes lignes

- Fonctionnel : fonctions anonymes, d'ordre supérieur ; “orienté valeur”
- Impératif : évaluer une expression produit une valeur et *a un effet*.
- Typé : les programmes mal typés sont rejetés *avant* l'exécution.

Les détails : MiniML

- Du polymorphisme de rang 1 (schémas de types).
- Des types enregistrement, des types sommes et du filtrage (match).
- Des références modifiables allouées dynamiquement.
- Des boucles dénombrées, “tant que” et “jusqu'à ce que”.

Le langage Hopix

Les grandes lignes

- Fonctionnel : fonctions anonymes, d'ordre supérieur ; “orienté valeur”
- Impératif : évaluer une expression produit une valeur et *a un effet*.
- Typé : les programmes mal typés sont rejetés *avant* l'exécution.

Les détails : MiniML

- Du polymorphisme de rang 1 (schémas de types).
- Des types enregistrement, des types sommes et du filtrage (match).
- Des références modifiables allouées dynamiquement.
- Des boucles dénombrées, “tant que” et “jusqu'à ce que”.

On va explorer le langage par parcours descendant de sa syntaxe abstraite.

Qu'est-ce qu'un programme Hopix ?

- Une liste de définitions de types et valeurs, et de déclaration externes.
- L'exécuter, c'est évaluer dans l'ordre les définitions de valeur.
- Les valeurs définies ne survivent pas à l'exécution du programme.

Qu'est-ce qu'un programme Hopix ?

- Une liste de définitions de types et valeurs, et de déclaration externes.
- L'exécuter, c'est évaluer dans l'ordre les définitions de valeur.
- Les valeurs définies ne survivent pas à l'exécution du programme.

type program = definition **list**

and definition =

```
| DefineValue of value_definition  
| DefineType of type_constructor  
    * type_variable list  
    * type_definition  
| DeclareExtern of identifier * type_scheme
```

Qu'est-ce qu'un programme Hopix ?

- Une liste de définitions de types et valeurs, et de déclaration externes.
- L'exécuter, c'est évaluer dans l'ordre les définitions de valeur.
- Les valeurs définies ne survivent pas à l'exécution du programme.

type program = definition **list**

and definition =

```
| DefineValue of value_definition  
| DefineType of type_constructor  
    * type_variable list  
    * type_definition  
| DeclareExtern of identifier * type_scheme
```

Un point de vue commun sur le typage

Le sens (“la sémantique”) d'un programme ne dépend pas des types.

Qu'est-ce qu'une définition de type ?

```
type type_definition =  
  (** A sum type for tagged values  
      [K1 (ty11, ..., ty1) | ... | K (ty1, ..., ty1)]. *)  
  | DefineSumType of (constructor * ty list) list  
  (** A record type { l1 : ty1, ..., l1 : ty }. *)  
  | DefineRecordType of (label * ty) list  
  (** A type with no visible definition. *)  
  | Abstract
```

- Des types somme (“algébriques”) récurifs.
- Des types enregistrements, formés par des listes de champs typés.
- Des types abstraits, c'est-à-dire sans définition.

Qu'est-ce qu'un type? Qu'un schéma de type?

```
type ty =  
  (** An instantiated type constructor [t <ty1, ..., ty >]. *)  
  | TyCon of type_constructor * ty list  
  (** A function type [ty1 → ty2]. *)  
  | TyArrow of ty * ty  
  (** A tuple type [ty1 * ... * ty ]. *)  
  | TyTuple of ty list  
  (** A type variable ['a]. *)  
  | TyVar of type_variable  
  
type type_scheme =  
  ForallTy of type_variable list * ty located
```

Qu'est-ce qu'un type? Qu'un schéma de type?

```
type ty =  
  (** An instantiated type constructor [t <ty1, ..., ty >]. *)  
  | TyCon of type_constructor * ty list  
  (** A function type [ty1 → ty2]. *)  
  | TyArrow of ty * ty  
  (** A tuple type [ty1 * ... * ty ]. *)  
  | TyTuple of ty list  
  (** A type variable ['a]. *)  
  | TyVar of type_variable
```

```
type type_scheme =  
  ForallTy of type_variable list * ty located
```

Le polymorphisme de rang 1

Les types de la forme $(\forall \alpha. \tau_1) \rightarrow \tau_2$ ne font pas partie du langage.

Qu'est-ce qu'une définition de valeur ?

```
type value_definition =  
  (** A toplevel definition for a value. *)  
  | SimpleValue of expression polymorphic_definition  
  (** A toplevel definition for mutually recursive functions. *)  
  | RecFunctions of function_definition polymorphic_definition list  
  
and 'a polymorphic_definition = identifier * type_scheme option * 'a
```

Qu'est-ce qu'une définition de valeur ?

```
type value_definition =  
  (** A toplevel definition for a value. *)  
  | SimpleValue of expression polymorphic_definition  
  (** A toplevel definition for mutually recursive functions. *)  
  | RecFunctions of function_definition polymorphic_definition list  
  
and 'a polymorphic_definition = identifier * type_scheme option * 'a
```

Remarques

- Les types ont été factorisés pour minimiser les répétitions.
- Cette factorisation des types se traduira en factorisation du code.

Qu'est-ce qu'une expression ? Logistique.

type expression = expression_body located

and expression_body =

(** A variable identifies a value. If this value is polymorphic,
it can be instantiated using a list of types. *)

| **Variable of** identifier located * ty located **list** option

(** A local definition of value(s) [value_definition; e₂]. *)

| **Define of** value_definition * expression

(** A type annotation [(e : ty)]. *)

| **TypeAnnotation of** expression * ty located

...

Qu'est-ce qu'une expression ? Logistique.

type expression = expression_body located

and expression_body =

(** A variable identifies a value. If this value is polymorphic,
it can be instantiated using a list of types. *)

| **Variable of** identifier located * ty located **list** option

(** A local definition of value(s) [value_definition; e₂]. *)

| **Define of** value_definition * expression

(** A type annotation [(e : ty)]. *)

| **TypeAnnotation of** expression * ty located

...

Remarques

- Je présente les constructions dans un ordre distinct du source.
- Les nœuds de l'arbre sont annotés avec des positions (*locations*).
 - Permet d'améliorer les messages d'erreur, p.ex., de typage.
 - La factorisation expression/body n'est pas dans hopixAST.ml.

Qu'est-ce qu'une expression ? Formes d'introduction.

```
...
(** A literal is a constant written "as is". *)
| Literal of literal located
(** A tagged value [K <ty1, ..., tym> (e1, ..., e )]. *)
| Tagged of constructor located
    * ty located list option
    * expression list
(** A record [{l1 = e1, ..., l = e } <ty1, ..., ty >]. *)
| Record of (label located * expression) list
    * ty located list option
(** A tuple [(e1, ..., en)]. *)
| Tuple of expression list
(** An anonymous function [ pattern -> e ]. *)
| Fun of function_definition
...
```

Qu'est-ce qu'une expression ? Formes d'élimination.

```
...
(** A record field access [e.l]. *)
| Field of expression * label located * ty located list option
(** A pattern matching [match (e) { p1 -> e1 | ... | p -> e }]. *)
| Case of expression * branch located list
(** A conditional expression [if (...) ... else ...]. *)
| IfThenElse of expression * expression * expression
(** A function application [a1 (a2)). *)
| Apply of expression * expression
...
```


Qu'est-ce qu'une expression ? Effets de bord.

```
...
(** A sequence [e1 ; ... ; e ] *)
| Sequence of expression list
(** A reference allocation. *)
| Ref of expression
(** An assignment. *)
| Assign of expression * expression
(** A dereference. *)
| Read of expression
(** An unbounded loop of the form [while (...) { ... }]. *)
| While of expression * expression
(** A bounded loop of the form [for x in (e1 to e2) { ... }]. *)
| For of identifier located * expression * expression * expression
```

Qu'est-ce qu'un motif?

and pattern =

```
(** A pattern which is simply an identifier. *)
| PVariable of identifier located
(** A wildcard pattern [_]. *)
| PWildcard
(** A pattern with a type annotation of type form [p : ty] *)
| PTypeAnnotation of pattern located * ty located
(** A literal pattern. *)
| PLiteral of literal located
(** A pattern for a tagged value K <ty1, ..., ty> (p1, ..., p). *)
| PTaggedValue of constructor located
                        * ty located list option * pattern located list
(** A pattern for a record {l1 = p1, ..., l = p } < ty1, ..., ty *)
| PRecord of (label located * pattern located) list * ty located list
(** A pattern for a tuple (p1, ..., p). *)
| PTuple of pattern located list
(** A disjunctive pattern [ p1 | ... | p ]. *)
| POr of pattern located list
(** A conjunctive pattern [ p1 & ... & p ]. *)
| PAnd of pattern located list
```

Synthèse

- Hopix est proche d'un langage fonctionnel des années $\sim$ 1980.
- On a découvert sa syntaxe abstraite sous forme de code OCaml.
- On aurait aussi pu la présenter sous la forme d'une grammaire BNF.

Synthèse

- Hopix est proche d'un langage fonctionnel des années $\sim$ 1980.
- On a découvert sa syntaxe abstraite sous forme de code OCaml.
- On aurait aussi pu la présenter sous la forme d'une grammaire BNF.

Syntaxe abstraite, syntaxe concrète

Le jalon 1 concerne la syntaxe *concrète*. En route !